

Day04 Part E 学习文档 v1.0 : Runtime 如何思考 (How Runtime Thinks) - Provider Adapter

本文是《从零实现 Agent Runtime》学习阶段的 Day04 Part E 正式学习文档。

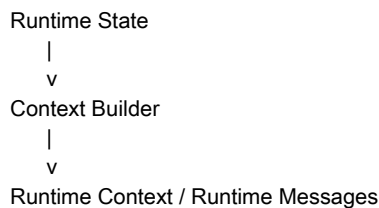
Part A 建立“Prompt 不是 Context”的世界观；Part B 解释 Runtime State 如何保存 Agent 的当前世界；Part C 解释 Context Builder 如何把 State 投影成 LLM 可见上下文；Part D 解释有限 Context Window 下如何做 Selection、Compression、Eviction 和 Assembly。Part E 补齐最后一层：Provider Adapter，回答 Runtime 如何不绑定任何一家 LLM Provider。

与上一节的联系

Part D 的核心结论是：

Context Window Management 是 Agent Runtime 的认知资源管理系统。它负责在有限 token 空间中完成 Selection、Allocation、Ranking、Eviction、Compression 和 Recovery，使 LLM 每一轮都看到当前任务最需要的信息。

Part D 之后，Runtime 已经能决定模型应该看到什么：



但这里还有一个工程问题：

Runtime 生成了统一上下文之后，应该如何发送给 OpenAI、Claude、Gemini 或本地模型？

如果 Runtime Core 直接调用某个供应商 SDK：

```
openai.chat.completions.create(...)
```

Runtime 就会被 OpenAI 的 Message Schema、Tool Calling 协议、Streaming Event、Error Format 和 Usage 字段绑定。模型一换，Context Builder、Tool Runtime、Streaming UI、Retry、Cost Tracking 都可能被迫改动。

所以 Part E 的核心结论是：

Provider Adapter 是 Runtime 和外部 LLM Provider 之间的边界层。Context Builder 决定“给模型看什么”，Provider Adapter 决定“如何让某个模型收到这些内容，并把模型返回结果翻译回 Runtime 世界”。

Day04 到这里可以串成：

Part A:
Prompt 不是 Context

Part B:
Runtime State 是 Agent 的世界

Part C:
Context Builder 把世界投影成 LLM 可见上下文

Part D:
Context Window Management 管理有限认知资源

Part E:
Provider Adapter 隔离模型供应商协议差异

Day05:
Tool Calling 进入执行系统

目录

Part E 总体目标

第一章：为什么 Runtime 不能绑定 LLM Provider

第二章：Unified Model Interface

第三章：Provider Adapter Architecture

第四章：Streaming / Tool Calling / Error / Usage Adapter

第五章：Mini Provider Adapter 实现设计

第六章：Day04 全部串联总结

Day04 Final Agent Runtime Architecture v3

前置问题回收

面试视角

写书 TODO

写书素材

本 Part 核心认知升级

下一节学习计划

本章思考题

Part E 总体目标

Provider Adapter 解决的不是“怎么调用某个模型 API”，而是：

如何让 Agent Runtime 拥有自己的模型抽象，而不是被某一家 Provider 的协议定义。

一个工业级 Agent Runtime 至少要回答五类问题：

Runtime 内部应该使用什么统一消息模型？

Tool Definition、Tool Call、Tool Result 如何跨 Provider 统一？

Streaming 事件如何统一成 Runtime Event？

Provider Error 如何映射成可重试、可降级、可观测的 Runtime Error？

Usage Metric 如何标准化，支撑成本统计、限流和监控？

Part E 的关键词是：

Dependency Inversion
Adapter Pattern
Unified Model Interface
Provider Protocol Isolation
Runtime Domain Model

LLM Provider 是外部能力，不应该反过来定义 Runtime 的内部世界。

第一章：为什么 Runtime 不能绑定 LLM Provider

1.1 初学 Agent 时的写法

很多最小 Demo 会这样写：

```
const response = await openai.chat.completions.create({
  model: "gpt-5",
  messages,
});
```

链路很短：

```
User
|
v
OpenAI SDK
|
v
Answer
```

如果目标只是一次普通 Chatbot 调用，这样足够。但如果目标是 Agent Runtime，这个设计很快会变成架构负债。

1.2 Runtime Core 不应该认识具体 Provider

Runtime Core 负责的是 Agent 内部世界：

```
Runtime State
Conversation
Memory
Workspace Index
Tool Result
Plan
Checkpoint
Context Builder
Token Budget
```

到 Context Builder 输出时，Runtime 已经完成了最重要的判断：

当前这一轮 LLM 应该看到什么。

下一步“怎么发送给具体模型”不应该由 Runtime Core 直接处理。如果 Runtime 里到处出现：

```
openai.xxx()
anthropic.xxx()
gemini.xxx()
```

就会造成基础设施协议污染业务内核。

1.3 绑定 Provider 的后果

如果一开始绑定 OpenAI，一年后因为成本、延迟、地区、合规或模型能力需要切到 Claude、Gemini、DeepSeek 或本地模型，可能需要连带修改：

- Context Builder ;
- Tool Calling ;
- Streaming ;
- Error Handling ;
- Retry ;

- Logging ;
- Cost Tracking ;
- 测试 Mock ;
- 前端事件消费。

这就是典型的依赖方向错误：

错误：

Runtime Core -> Provider SDK

正确：

Runtime Core -> LLMProvider Interface <- Provider Adapter

1.4 Provider Adapter 的核心思想

Provider Adapter 和前端/后端常见 Adapter Pattern 是同一套思想。

业务代码不应该直接散落：

```
uploadToOSS(file);
uploadToS3(file);
uploadToCOS(file);
```

而应该依赖：

```
interface Storage {
  upload(file: File): Promise<UploadResult>;
}
```

Agent Runtime 也一样：

```
Agent Runtime
  |
  v
LLMProvider Interface
  |
  v
Provider Adapter
  |
+-----+-----+-----+
|         |         |         |
OpenAI  Claude Gemini
```

Runtime 只认识能力接口，不认识供应商协议。

第二章：Unified Model Interface

2.1 错误设计：以 Provider 为中心

错误的起点通常是：

```
type Message = OpenAI.ChatCompletionMessage;
```

然后 Runtime 内部到处出现：

```
message.tool_calls;
message.function_call;
response.choices[0].message;
usage.prompt_tokens;
```

这样写出来的不是 Agent Runtime，而是 OpenAI Runtime。

当 Claude 或 Gemini 接入时，代码会开始扩散：

```

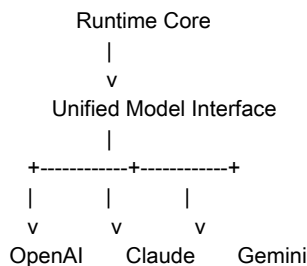
if (provider === "openai") {
  // ...
} else if (provider === "claude") {
  // ...
} else if (provider === "gemini") {
  // ...
}

```

Provider 差异从边界层泄漏到核心层，系统会逐步腐化。

2.2 正确设计：Runtime 是中心

正确方向是 Runtime 定义自己的领域模型：



Runtime 定义 Agent 世界里的语言；Provider Adapter 负责翻译。

2.3 LLMPProvider

Runtime 中的模型能力接口可以从两个核心方法开始：

```

export interface LLMPProvider {
  chat(request: ChatRequest): Promise<ChatResponse>;
  stream(request: ChatRequest): AsyncIterable<RuntimeEvent>;
}

```

这个接口描述的是：

LLM 在 Runtime 中应该具备什么能力。

它不应该暴露 OpenAI、Claude 或 Gemini 的 SDK 类型。

2.4 ChatRequest

Runtime 调模型时真正需要的是：

```

export interface ChatRequest {
  messages: RuntimeMessage[];
  tools?: ToolDefinition[];
  options?: ModelOptions;
}

```

其中 model 也可以存在，但最好表达为 Runtime 的能力需求，而不是具体供应商字段。例如：

```

model: "fast" | "reasoning" | "cheap";

```

Provider Adapter 再把它映射成：

```

reasoning -> gpt-5-thinking / claude-opus / gemini-pro
fast      -> gpt-5-mini / claude-haiku / gemini-flash

```

2.5 RuntimeMessage

RuntimeMessage 是 Runtime 自己的消息模型：

```

export type MessageRole =
  | "system"
  | "user"
  | "assistant"

```

```

| "tool";

export interface RuntimeMessage {
  role: MessageRole;
  content: string | ContentBlock[];
  metadata?: {
    timestamp?: number;
    source?: string;
  };
}

```

它表达的是 Agent 内部的消息语义，而不是某个 Provider 的 DTO。

2.6 ContentBlock

现代 Agent Message 不只是字符串。它可能包含：

- text ;
- image ;
- file ;
- tool_call ;
- tool_result ;
- citation ;
- code block。

所以更现代的模型是：

```

export type ContentBlock =
  | TextBlock
  | ImageBlock
  | ToolCallBlock
  | ToolResultBlock
  | FileBlock;

export interface TextBlock {
  type: "text";
  text: string;
}

export interface ToolCallBlock {
  type: "tool_call";
  id: string;
  name: string;
  arguments: unknown;
}

```

这个设计类似 React 从：

```
children: string
```

升级到：

```
children: ReactNode
```

因为内容形态变复杂后，领域模型必须能承载结构化内容。

2.7 ToolDefinition

工具也不应该使用 Provider 类型，而应该定义 Runtime 自己的工具模型：

```

export interface ToolDefinition {
  name: string;
  description: string;
}

```

```
    inputSchema: JSONSchema;
}
```

Runtime 只关心：

工具名
工具描述
参数结构

OpenAI、Claude、Gemini 如何表达这个工具，是 Adapter 的职责。

2.8 ChatResponse

响应也必须回到 Runtime 世界：

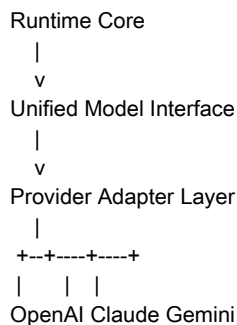
```
export interface ChatResponse {
  message: RuntimeMessage;
  usage?: Usage;
  finishReason?: FinishReason;
}
```

Runtime State 保存的是 RuntimeMessage，不是 OpenAIResponse。

第三章：Provider Adapter Architecture

3.1 Provider Adapter 在 Runtime 中的位置

完整链路是：



关键边界：

Runtime Core 之上：
Agent 自己的世界

Provider Adapter 之下：
外部模型生态

3.2 Adapter 不应该只是一个类

初级设计可能只有：

```
class OpenAIAdapter {
  chat() {}
}
```

工业 Runtime 通常会拆成多个转换器：

```
OpenAIAdapter
|
+-- MessageConverter
+-- ToolConverter
+-- StreamConverter
+-- ErrorMapper
+-- UsageMapper
```

+-- ResponseConverter

原因是 Provider 差异分布在多个维度，并不是一个 JSON 转换函数可以解决。

3.3 Message Converter

Message Converter 负责：

RuntimeMessage -> ProviderMessage

例如 Runtime：

```
{
  "role": "user",
  "content": [
    { "type": "text", "text": "hello" }
  ]
}
```

OpenAI 可能需要：

```
{
  "role": "user",
  "content": "hello"
}
```

Claude 可能需要：

```
{
  "role": "user",
  "content": [
    { "type": "text", "text": "hello" }
  ]
}
```

3.4 Tool Converter

Tool Converter 负责：

Runtime ToolDefinition -> Provider Tool Schema

Runtime：

```
{
  name: "search",
  description: "search web",
  inputSchema: { type: "object" }
}
```

OpenAI 风格：

```
{
  "type": "function",
  "function": {
    "name": "search",
    "description": "search web",
    "parameters": { "type": "object" }
  }
}
```

Claude 风格：

```
{
  "name": "search",
  "description": "search web",
  "input_schema": { "type": "object" }
}
```

Runtime 不应该知道这些差异。

3.5 Stream Converter

Streaming 是前端和 Agent 产品里最容易暴露 Provider 差异的地方。

OpenAI 可能返回：

```
response.delta
response.delta
response.done
```

Claude 可能返回：

```
content_block_start
content_block_delta
message_stop
```

Runtime 应该统一成：

```
export type RuntimeEvent =
  | { type: "text_delta"; text: string }
  | { type: "tool_call_start"; id: string; name: string }
  | { type: "tool_call_delta"; id: string; argumentsDelta: string }
  | { type: "usage"; usage: Usage }
  | { type: "finish"; reason?: FinishReason };
```

前端 UI 只消费 RuntimeEvent，不关心底层模型是 GPT、Claude 还是 Gemini。

3.6 Error Mapper

Provider 错误语义也不同。

OpenAI 可能返回 rate_limit，Claude 可能返回 overloaded，Gemini 可能返回不同的 quota 或 safety error。

Runtime 需要统一：

```
export interface RuntimeError {
  code:
    | "RATE_LIMIT"
    | "TIMEOUT"
    | "AUTH_ERROR"
    | "MODEL_UNAVAILABLE"
    | "SAFETY_BLOCKED"
    | "UNKNOWN";
  retryable: boolean;
  message: string;
  provider?: string;
}
```

这样 Runtime 才能统一做：

- retry ;
- fallback ;
- alert ;
- logging ;
- observability ;
- user-facing error message。

Adapter 不只是转换数据，它还负责转换语义。

3.7 Usage Mapper

Usage Metric 也需要统一。

不同 Provider 字段可能是：

OpenAI:
prompt_tokens / completion_tokens

Claude:
input_tokens / output_tokens

Gemini:
promptTokenCount / candidatesTokenCount

Runtime 应该使用：

```
export interface Usage {  
  inputTokens: number;  
  outputTokens: number;  
  totalTokens: number;  
  provider?: string;  
  model?: string;  
}
```

统一 Usage 是后续计费、限流、预算管理和监控的基础。

3.8 Strategy Pattern 与依赖倒置

Runtime 构造时依赖接口：

```
class AgentRuntime {  
  constructor(private llm: LLMProvider) {}  
}
```

运行时注入实现：

```
new AgentRuntime(new OpenAIAdapter());  
new AgentRuntime(new ClaudeAdapter());  
new AgentRuntime(new MockProvider());
```

这就是 Strategy Pattern，也体现 Dependency Inversion：

高层 Runtime Core 不依赖低层 Provider SDK。
低层 Adapter 反过来实现高层定义的 LLMProvider 接口。

Mock Provider 很重要，因为 Agent Runtime
测试不能依赖真实模型，否则测试会慢、贵、不稳定、不可复现。

第四章：Streaming / Tool Calling / Error / Usage Adapter

4.1 为什么这一章重要

如果只是普通聊天，Adapter 可能只需要 Message 转换。

但 Agent 不一样。Agent 会涉及：

```
LLM  
|  
+-- Streaming  
+-- Tool Calling  
+-- Structured Output  
+-- Usage  
+-- Error  
+-- Retry  
+-- Fallback
```

Provider Adapter 实际上是：

Runtime 与模型生态之间的防火墙。

4.2 Streaming Adapter

普通响应是一次性返回完整结果：

Request -> LLM -> Full Response

Streaming 是逐步返回：

```
text_delta
text_delta
tool_call_delta
finish
```

Runtime 应该定义自己的 Event Model：

```
type RuntimeEvent =
  | TextDeltaEvent
  | ToolCallStartEvent
  | ToolCallDeltaEvent
  | UsageEvent
  | FinishEvent;
```

这样前端可以只写：

```
useRuntimeEvent((event) => {
  if (event.type === "text_delta") {
    append(event.text);
  }
});
```

UI 不需要分辨 OpenAI、Claude、Gemini 的事件细节。

4.3 Tool Calling Adapter

Tool Calling 是 Day05 的核心，但 Part E 先建立 Adapter 视角。

Runtime 世界里的 Tool：

```
type ToolDefinition = {
  name: string;
  description: string;
  inputSchema: JSONSchema;
};
```

Provider Adapter 负责双向转换：

```
Runtime ToolDefinition
  |
  v
Provider Tool Schema

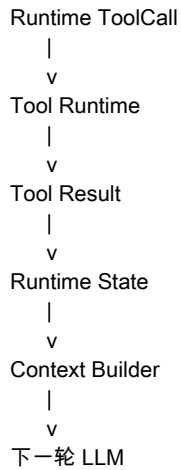
Provider Tool Call
  |
  v
Runtime ToolCall
```

当模型返回工具调用时，Runtime 不应该保存 Provider 原始结构，而应该转成：

```
export interface ToolCall {
  id: string;
  name: string;
  arguments: unknown;
}
```

然后进入：

```
Provider Adapter
  |
  v
```



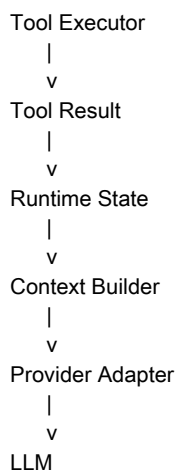
关键认知：

LLM 负责产生 Tool Call 意图，真正执行 Tool 的是 Runtime。

4.4 Tool Result Adapter

Tool Result 也不是直接塞回 LLM。

正确路径是：

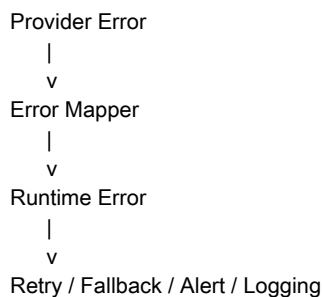


Tool Result 是否进入下一轮 Context、以什么形式进入、是否压缩或摘要，属于 Context Builder 和 Context Window Management 的职责。

4.5 Error Adapter

模型服务一定会失败。Runtime 不能让每个 Provider 的错误结构进入核心逻辑。

Adapter 应该把 Provider Error 映射为 Runtime Error：



这会影响系统稳定性，而不只是代码整洁。

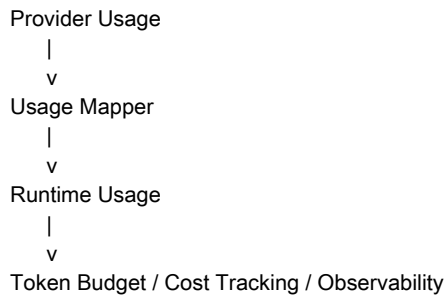
4.6 Usage Adapter

Agent 平台必须知道：

- 当前调用消耗多少 token；
- 大概花了多少钱；
- 用户额度是否足够；
- 是否触发限流；
- 哪个模型成本更高；
- Context Window 策略是否需要调整。

所以 Usage 需要从 Provider 字段归一化到 Runtime 字段。

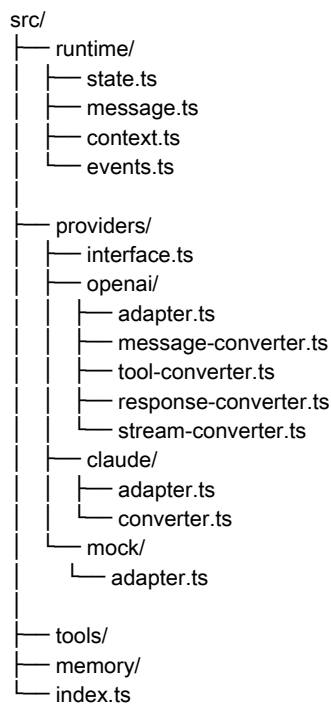
Usage Mapper 连接 Part D 的 Token Budget：



第五章：Mini Provider Adapter 实现设计

5.1 目录结构

如果在 mini-agent-runtime 中实现 Provider Adapter，目录可以这样组织：



这里体现的边界是：

Runtime Domain
和
Provider Infrastructure
完全隔离

5.2 Runtime 层先定义自己的模型

不要从 OpenAI SDK 类型开始，而要从 Runtime Domain 开始：

```
export type MessageRole =  
  | "system"  
  | "user"  
  | "assistant"  
  | "tool";  
  
export interface RuntimeMessage {  
  role: MessageRole;  
  content: ContentBlock[];  
  metadata?: {  
    timestamp?: number;  
    source?: string;  
  };  
};  
}
```

5.3 Provider Interface

providers/interface.ts 是 Provider Layer 的核心：

```
export interface LLMPProvider {  
  chat(request: ChatRequest): Promise<ChatResponse>;  
  stream(request: ChatRequest): AsyncIterable<RuntimeEvent>;  
}
```

Runtime 只依赖这个接口。

5.4 OpenAI Adapter

OpenAIAdapter 实现接口，但不把 OpenAI 类型暴露给 Runtime：

```
export class OpenAIAdapter implements LLMPProvider {  
  constructor(private client: OpenAI) {}  
  
  async chat(request: ChatRequest): Promise<ChatResponse> {  
    const messages = convertMessages(request.messages);  
    const tools = convertTools(request.tools);  
  
    const response = await this.client.chat.completions.create({  
      messages,  
      tools,  
    });  
  
    return convertResponse(response);  
  }  
}
```

核心流程：

```
RuntimeMessage  
  |  
  v  
MessageConverter  
  |  
  v  
OpenAI Message  
  |  
  v  
OpenAI API  
  |  
  v  
ResponseConverter  
  |  
  v  
RuntimeMessage
```

5.5 Mock Provider

Mock Provider 是测试 Runtime 的基础：

```
export class MockProvider implements LLMPProvider {
  async chat(): Promise<ChatResponse> {
    return {
      message: {
        role: "assistant",
        content: [
          {
            type: "text",
            text: "mock answer",
          },
        ],
      },
    };
  }
}
```

这样可以稳定测试：

- Runtime State ;
- Context Builder ;
- Tool Loop ;
- State Transition ;
- Error Recovery ;
- Streaming UI。

5.6 Mini Runtime 已经具备什么

做到这里，一个最小 Runtime 已经具备雏形：

```
Runtime State
+
Context Builder
+
RuntimeMessage
+
LLMPProvider Interface
+
Provider Adapter
+
RuntimeEvent
+
Mock Provider
```

下一步 Day05 接入 Tool Runtime 后，Agent 才真正从“会思考”进入“能行动”。

第六章：Day04 全部串联总结

Day04 完成的是 Agent Runtime 的“大脑架构”。

从 Day01 到 Day03，我们在回答：

什么是 Agent ?
Runtime 是什么 ?
为什么需要 Runtime ?

Day04 开始回答：

Runtime 如何让 LLM 看见世界、理解当前任务，并在不同模型之间保持稳定？

6.1 Part A : Prompt 不是 Context

最初的错误理解是：

Context = Prompt

Part A 的升级是：

Prompt 只是 Runtime State 的一次投影结果。

Context 不是手写字符串，而是 Runtime 为当前模型调用构造的可见世界。

6.2 Part B : Runtime State

Agent 不是一次请求，而是持续运行过程。Runtime State 保存：

- Conversation ；
- Goal ；
- Plan ；
- Tool Result ；
- Memory ；
- Workspace ；
- Checkpoint ；
- Metadata。

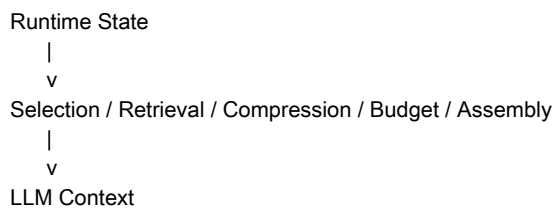
核心升级：

Conversation 记录发生过什么。

Runtime State 描述现在是什么。

6.3 Part C : Context Builder

Runtime State 不能直接发送给 LLM。Context Builder 负责：



核心升级：

Prompt Engineering -> Context Engineering

6.4 Part D : Context Window Management

当信息超过 Context Window 时，不是简单截断，而是认知资源管理：

- Token Budget ；
- Priority Ranking ；
- Context Eviction ；
- Context Compression ；
- Summary Memory ；
- Hierarchical Context ；

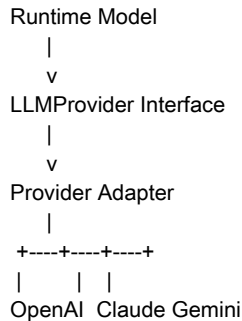
- Recovery。

核心升级：

让模型看到最多信息，不等于让模型做出最好判断。

6.5 Part E：Provider Adapter

Provider Adapter 让 Runtime 不绑定任何模型供应商：



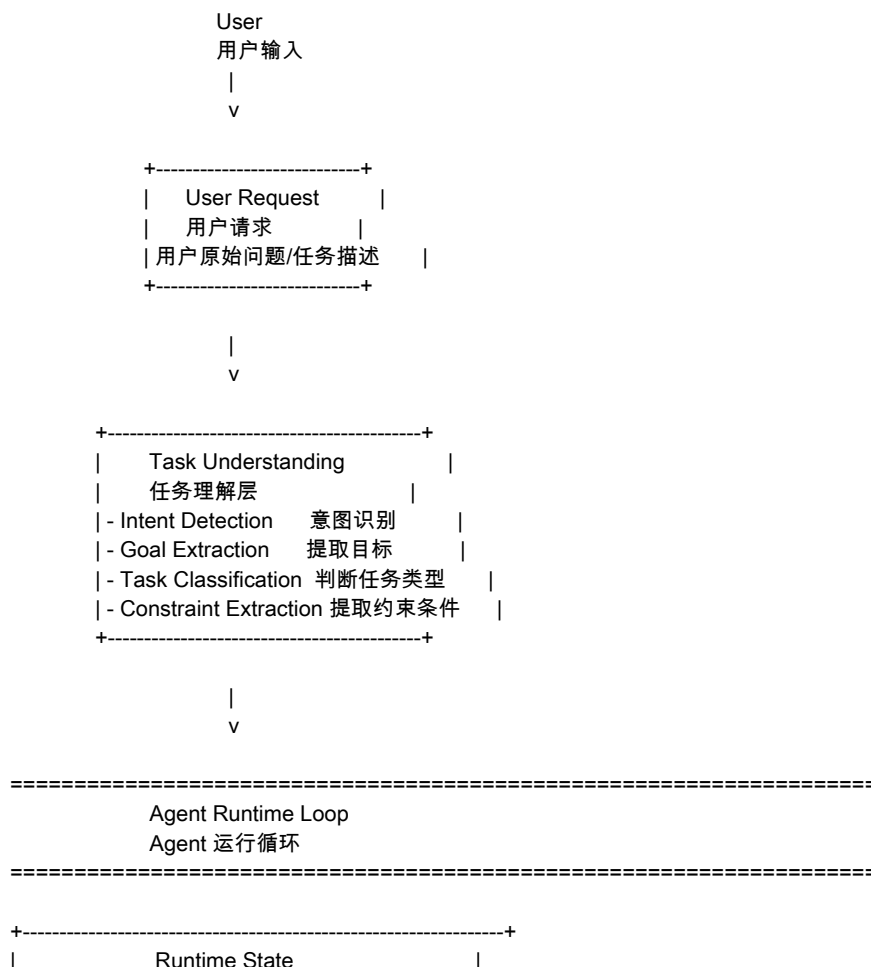
核心升级：

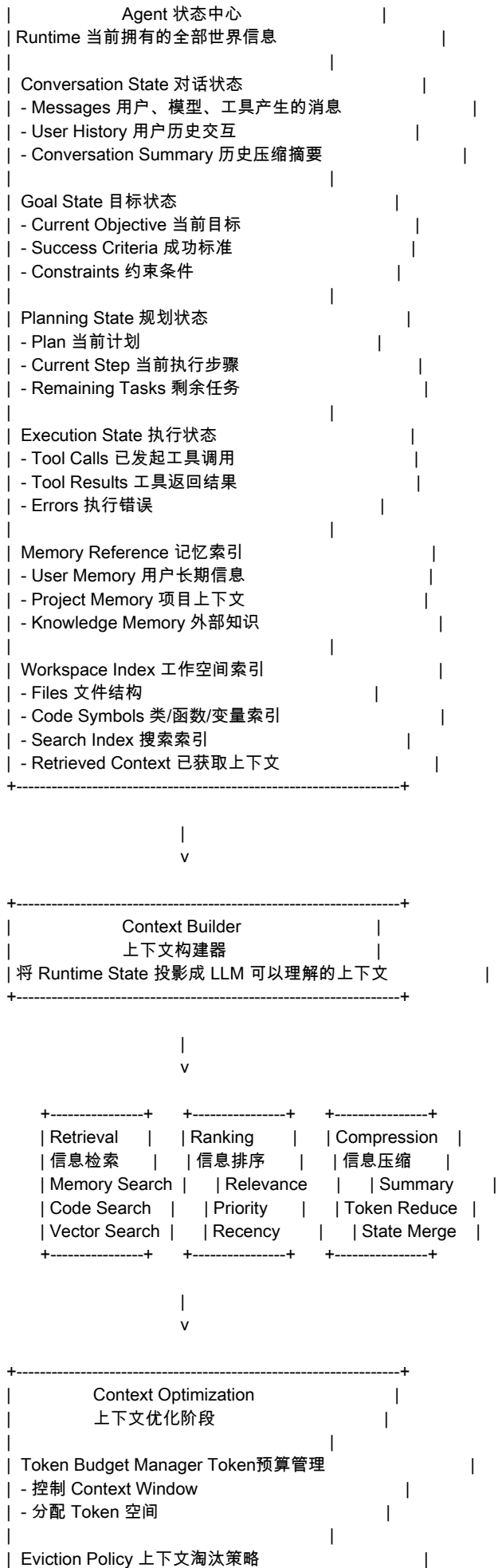
LLM 是可替换推理引擎。

Runtime 才是 Agent 的稳定内核。

Day04 Final Agent Runtime Architecture v3

这是本次学习最后校正后的“学习版”总架构图。它保留英文工程术语，同时增加中文职责说明，并补回 Part D 中非常关键的 Compression、Eviction、Assembly，以及 Runtime State 中的 Workspace Index。





- 删除低价值信息	
- 移除过期历史	
- 去重重复上下文	
+-----+	

|
v

+-----+	
Context Assembly	
上下文组装	
最终生成发送给 LLM 的输入	
+-----+	
- System Prompt	系统规则
- Goal	当前目标
- Constraints	约束条件
- History	相关历史
- Memory	相关记忆
- Evidence	检索证据
- Tool Result	工具结果
+-----+	

|
v

+-----+	
Runtime Model	
Runtime 内部统一模型	
屏蔽外部 Provider 差异	
+-----+	
- RuntimeMessage	统一消息模型
- ContentBlock	内容块
- ToolDefinition	工具定义
- ToolCall	工具调用
- RuntimeEvent	运行事件
+-----+	

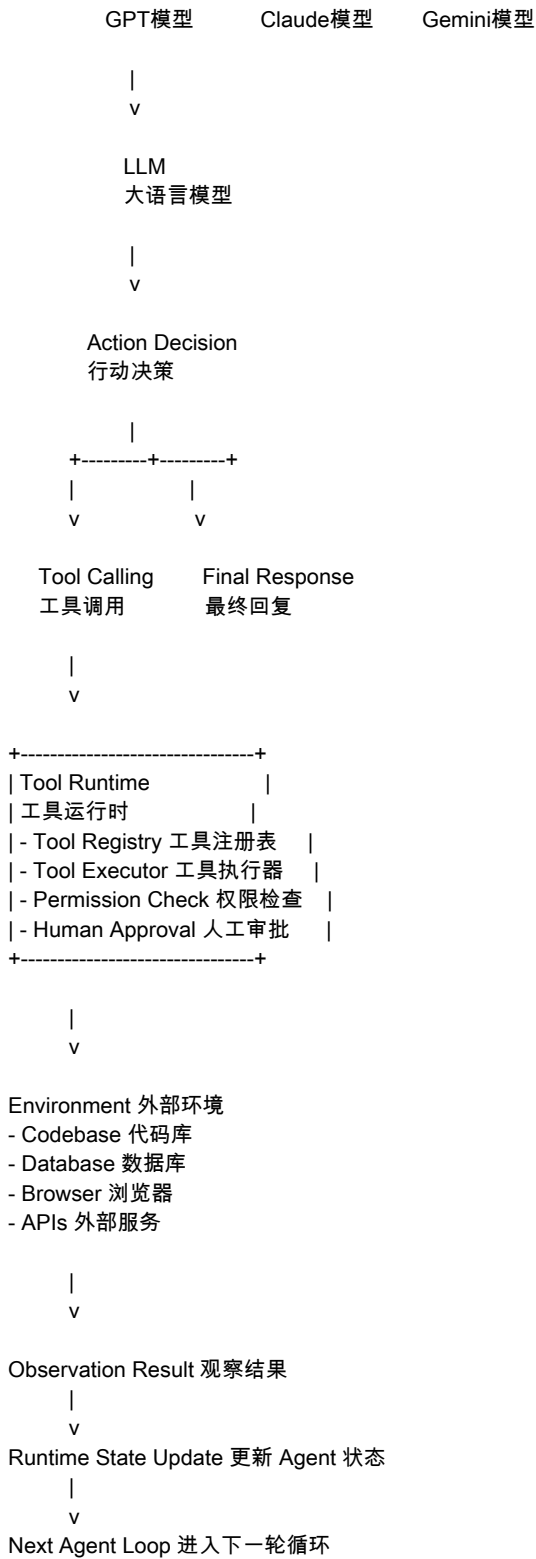
|
v

+-----+	
LLM Provider Interface	
模型能力接口	
Runtime 定义模型需要提供什么能力	
+-----+	
- chat()	普通调用
- stream()	流式调用
+-----+	

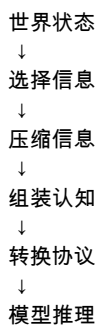
|
v

+-----+	
Provider Adapter	
模型适配层	
将 Runtime Model 转换成具体模型协议	
+-----+	
- Message Converter	消息转换
- Tool Converter	工具格式转换
- Stream Converter	流事件转换
- Error Mapper	错误统一
- Usage Mapper	Token统计统一
+-----+	

v	v	v
OpenAI	Claude	Gemini



这张图的核心不是模块名，而是数据变化链路：



↓
产生行动
↓
改变世界
↓
更新状态

前置问题回收

Q1：为什么 Agent = LLM + Workflow + Tools + Memory 比单纯 LLM 更接近企业落地？

因为企业需要的不是一个会聊天的模型，而是一个运行在业务流程中的自动化系统。

```
Agent Runtime
|
+-- State
+-- Tools
+-- Memory
+-- Workflow
|
v
Context Builder
|
v
Provider Adapter
|
v
LLM
```

LLM 只是推理部分，Runtime 才负责状态、行动、流程和连续性。

Q2：为什么企业 Agent 往往需要 Workflow？

企业流程通常有大量确定性步骤。例如客服订单场景：

```
识别用户身份
|
v
查询订单
|
v
判断状态
|
v
生成回复
```

其中 70%-80% 是 Workflow，LLM 负责自然语言理解、意图判断和表达生成。未来企业 Agent 大概率是：

Workflow + LLM + Tools

而不是完全自由 Agent。

Q3：为什么 Agent Runtime 很像操作系统？

对应关系：

| OS | Agent Runtime | | Process | Agent Run | | Memory | Runtime State | | Scheduler | Runtime Loop | | IO | Tool Calling | | Memory Management | Context Management |

Agent Runtime 也需要管理执行、状态、IO、资源和错误恢复。

面试视角

1. 什么是 Agent Runtime ?

高级回答：

Agent Runtime 是负责管理 Agent 生命周期的执行环境，它负责维护状态、构建上下文、调度模型调用、执行工具、处理事件和错误恢复。LLM 只是 Runtime 中负责推理的一部分。

2. 为什么需要 Context Builder ?

回答：

Runtime 中的信息量通常超过模型 Context Window，因此需要 Context Builder 根据当前任务动态选择、压缩和组织信息，将 Runtime State 投影成模型可理解的上下文。

3. Provider Adapter 解决什么问题？

回答：

Provider Adapter 将 Runtime 内部统一模型和不同 LLM Provider 的协议隔离，包括 Message Schema、Tool Calling、Streaming Event、Error Format 和 Usage Metric，使 Runtime 不绑定任何具体模型。

4. Tool Calling 是 LLM 做的吗？

高级回答：

LLM 负责产生 Tool Call 意图，但真正的工具执行属于 Runtime。Runtime 接收模型产生的 Tool Call，然后调用 Tool Executor，将结果重新写入 State，并继续下一轮推理。

写书 TODO

Day04 可以沉淀为一章：

Chapter：Building Agent Runtime Core

- 4.1 Why Agent Needs Runtime
- 4.2 Runtime State Management
- 4.3 Context Engineering
- 4.4 Context Window Management
- 4.5 Provider Abstraction
- 4.6 Runtime Architecture Design
- 4.7 Build Mini Agent Runtime

写作重点：

- 不要把 Agent 写成 LLM API 包装器；
- Runtime State 是 Agent 的世界模型；
- Context Builder 是从世界到认知的投影器；
- Context Window Management 是认知资源管理；
- Provider Adapter 是模型生态兼容层；
- Tool Runtime 是 Day05 从思考进入行动的关键。

写书素材

素材 1：《LLM 是 Agent 的大脑，但 Runtime 才是生命系统》

很多人认为：

Agent = LLM

更准确是：

Agent = Runtime + LLM

LLM 提供推理，Runtime 提供记忆、状态、行动和生命周期。

素材 2：《Context Engineering：新时代 Agent 工程能力》

Prompt Engineering 问的是：

我要告诉模型什么？

Context Engineering 问的是：

这一刻模型应该知道什么？

区别是从写提示词升级到设计信息流。

素材 3：《Provider Adapter：Agent 世界的浏览器兼容层》

Web 世界：

```
Browser API
|
v
Chrome / Safari / Firefox
```

Agent 世界：

```
LLMProvider Interface
|
v
GPT / Claude / Gemini
```

Provider Adapter 对 Agent Runtime 的意义，就像兼容层对 Web Runtime 的意义。

本 Part 核心认知升级

1. Agent 不等于 LLM

以前：

调用模型 = Agent

现在：

Agent = Runtime + LLM

2. Runtime State 是 Agent 的世界

以前：

Prompt 保存上下文

现在：

State 保存世界
Context 保存投影

3. Context Builder 是 Agent 的认知过滤器

以前：

Prompt 拼接

现在：



4. Provider Adapter 是架构边界

以前：

换模型 = 改 Runtime

现在：

换模型 = 换 Adapter

5. Runtime 决定真实表现

模型决定推理能力的上限，Runtime 决定 Agent 在真实世界中的表现。Runtime 让模型拥有状态、记忆、工具、事件、恢复能力和可替换模型边界。

下一节学习计划

Day05 进入 Tool Calling。

预计主题：

- Part A：Tool Calling 基础模型
- Part B：LLM 如何决定调用 Tool
- Part C：Tool Schema 设计
- Part D：Tool Registry
- Part E：Tool Executor
- Part F：Tool Result 回流 Runtime
- Part G：Multi Tool Loop
- Part H：Human Approval
- Part I：Mini Tool Runtime 实现

Day04 建立的是 Agent 的认知系统；Day05 开始进入执行系统，回答：

Agent 如何从思考走向行动。

本章思考题

为什么 RuntimeMessage 不应该直接复用 OpenAI 的 Message 类型？

如果 OpenAI 明天调整 Tool Calling 协议，一个设计良好的 Runtime 为什么不需要大规模修改？

Context Builder 和 Provider Adapter 的边界分别是什么？

Streaming Event 为什么需要 Runtime 自己的统一事件模型？

为什么 Mock Provider 是 Agent Runtime 测试体系的必要部分？

Workspace Index 为什么应该属于 Runtime State？

Context Builder 为什么必须包含 Retrieval、Ranking、Compression、Eviction、Token Budget 和 Assembly ?